REDUCTION MODULO BINARY POLYNOMIALS WITH LOGARITHMIC FEEDBACK DEPTH

JUNYU ZHOU AND KAIYI ZHANG

Abstract. Polynomial modular reduction is central to binary finite-field arithmetic and repeated Frobenius powering. Top-down shift/XOR folding can use few operations when the nonleading support is sparse, but a tap near the leading term creates a long feedback chain. We formulate this recurrence as inversion of a nilpotent shift operator and factor its inverse by characteristic-two Frobenius powers. The resulting *Frobenius-factorized reduction* (FFR) applies to every monic binary modulus without materializing a reciprocal or dense reduction matrix, and its shifts can be generated online without a persistent modulus-specific schedule. For degree m and nonempty nonleading support of size s, with nearest-tap distance $\Delta_{\min}$, FFR uses exactly $\lceil \log_2(m/\Delta_{\min}) \rceil$ Frobenius feedback stages and has scheduled work $O(ms(1 + \log(m/s)))$. A portable-C evaluation on 1,096 supports through degree 131072 identifies distinct FFR, López–Dahab, and gf2x-backed Barrett regions. On four certified irreducible moduli, FFR makes complete Rabin irreducibility testing 1.35–8.04 times faster than NTL and 1.60–6.81 times faster than the matched Barrett implementation.

1. Introduction

Arithmetic in a polynomial-basis representation of a binary extension field reduces products and squares modulo a monic polynomial. The same operation appears in polynomial factorization, irreducibility testing, and searches for sparse irreducible or primitive polynomials. In these settings the modulus is public and often fixed for many reductions, but the useful degree, support, and setup-amortization regimes vary substantially.

In the regime studied here, three modulus parameters provide the principal coordinates for reduction time:

(1) the degree m, which fixes the operand length and hence the amount of word-level data processed;
(2) the Hamming weight h, including the leading term, which determines the number of nonleading taps processed by tapwise shift/XOR methods; and
(3) the nearest-tap distance $\Delta_{\min}$, namely the gap from degree m to the highest positive nonleading tap, when one exists, which controls how far feedback can propagate before leaving the degree-m state; without such a tap there is no feedback chain.

Observed runtime also depends on complete tap placement, word alignment, setup, and the multiplication backend. The three parameters above separate the main

2020 *Mathematics Subject Classification.* Primary 11T06; Secondary 12-08, 68W10, 68W40.
Key words and phrases. binary polynomials, modular reduction, finite fields, sparse polynomials, Frobenius map, parallel algorithms.

size, sparsity, and feedback effects and therefore define the axes and slices used in our evaluation.

When the nonleading part is sparse, top-down folding uses little setup and can be extremely effective. Its work and latency, however, are controlled by different geometric features. Hamming weight controls how many shifts and XORs one fold introduces, whereas the distance from the leading term to the nearest internal tap controls how far feedback can propagate. Equal-degree, equal-weight moduli may therefore induce very different dependency chains.

Barrett and Montgomery reduction avoid this particular serial recurrence, but introduce multiplication kernels, modulus-dependent constants, and different setup costs. Reciprocal methods and dense linear maps expose additional parallelism, but do not automatically preserve the original tap support. We ask whether the tap-induced feedback chain can be shortened without first materializing a dense reciprocal or reduction matrix.

We answer this question with *Frobenius-factorized reduction* (FFR). It requires no materialized reciprocal, dense reduction matrix, or persistent modulus-specific shift schedule: the doubled shifts may be derived online from the original taps or retained in a lightweight reusable plan. Online execution uses an m-bit state and tap-sized descriptor scratch; "no persistent schedule" refers specifically to precomputed reduction data. For an input of degree below twice the modulus degree, reduction gives a finite forced recurrence. We write it as $(I + U)Y = H$ for a truncated shift operator U. When $U \neq 0$, characteristic two gives

$$(I+U)^{-1} = \prod_{k=0}^{r-1} (I + U^{2^k}), \qquad r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil, \tag{1}$$

and $U^{2^r} = 0$; when $U = 0$, no feedback stage is needed. Each Frobenius power contains only the active original taps, with their distances doubled.

The finite geometric inverse underlying (1) is classical [2, 6], but the identity alone does not give a support-nondensifying executable reducer: expanding it exposes mixed feedback paths and supplies neither a support-sensitive execution schedule nor packed-word work bounds. The algorithmic point here is that the Frobenius factors preserve the tap structure without combinatorial densification inside a stage; mixed paths arise only through composition across stages. This turns the inverse into an executable reducer for every monic binary modulus whose feedback depth, work, schedule size, and packed-word execution can be analyzed directly from the support geometry.

This construction generalizes our earlier Suwako reducer for trinomials $x^m + x^t + 1$ [1]. That work established the logarithmic-depth, precomputation-free trinomial case through two specialized doubling folds. With multiple taps, one cannot apply the trinomial fold to each tap independently: all shifts in one factor must read the same stage input, while their compositions create mixed paths between factors. The present contribution resolves this multi-tap execution and gives the resulting all-tap geometry, packed-word analysis, planned and online implementations, and comparative evaluation.

Our contributions are threefold.

(1) We derive a nilpotent-feedback formulation whose characteristic-two factorization introduces no within-factor support densification and gives a correct FFR algorithm for any monic binary modulus.

(2) We give exact tap-geometry analyses of coefficient work, packed-word work, feedback depth, schedule size, and workspace, together with planned and schedule-free online realizations.
(3) We provide a reproducible portable-C evaluation that maps winning and losing regions against serial folding, López–Dahab, and gf2x-backed Barrett reduction, and test the reduction advantage inside a complete Rabin irreducibility test with an optimized NTL baseline.

Correctness holds for every monic binary modulus, while sparsity governs the performance regime. Our depth results use the synchronous feedback-stage and bounded-fan-in models defined below; complete-workload effects are evaluated separately through Rabin irreducibility testing.

Sections 2–3 fix the problem and comparison models. Sections 4–6 develop the method and analysis. Section 7 states the prior-art boundary. Section 8 contains the implementation and application evidence, and Section 9 concludes.

2. Preliminaries and Cost Models

Let

$$g(x) = x^m + q(x) = x^m + \bigoplus_{t\in T} x^t \in \mathbb{F}_2[x], \qquad T \subseteq \{0,\dots,m-1\}, \tag{2}$$

be monic, and set $h = 1 + |T|$. For $t \in T$, define $\Delta_t = m - t$. If $T \neq \varnothing$, let

$$\Delta_{\min} = \min\{\Delta_t : t \in T\}. \tag{3}$$

When $T \subseteq \{0\}$, the feedback operator below is zero and we set $r = 0$; otherwise r is defined by (9).

Identify degree-below-m polynomials with vectors in $\mathbb{F}_2^m$. Let N be the truncated right shift

$$(N^d y)_i = \begin{cases} y_{i+d}, & 0 \le i < m-d, \\ 0, & m-d \le i < m, \end{cases} \qquad N^m = 0. \tag{4}$$

For $A \in \mathbb{F}_2[x]$ with $\deg A < 2m$, write

$$A = L + x^m H, \qquad \deg L, \deg H < m. \tag{5}$$

For degree-below-m input Y, define linear maps V and U by

$$\begin{aligned} qY &= V(Y) + x^m U(Y), \\ V(Y) &= \sum_{t\in T} (x^t Y \bmod x^m), \qquad & U &= \sum_{t\in T} N^{\Delta_t}. \end{aligned} \tag{6}$$

Here and below an empty sum denotes the zero map. The term $t = 0$, when present, contributes the identity to V and the zero shift N^m to U. Monicity is sufficient for a unique remainder; irreducibility is required only when the quotient is intended to be a field.

We distinguish scheduled coefficient work, machine-word work, feedback depth, bounded-fan-in gate depth, setup, and temporary space. Scheduled coefficient work counts shifted coefficient contributions, while machine-word work counts the packed source-level data path. Machine latency and throughput are measured under a fixed platform and timed boundary. For the packed implementation, coefficient i is bit $i \bmod W$ of word $\lfloor i/W \rfloor$ in a little-endian array of $n = \lceil m/W \rceil$ words. An input representing $\deg A < 2m$ has canonical zero padding above coefficient

TABLE 1. Applicability of the compared direct-reduction methods. Throughout, g is as in (2), $\deg A < 2m$, and $n = \lceil m/W \rceil$; the dense-map bound is the 64 MiB limit used in the evaluation.

Method	FFR	Serial	BarrettGF2X	López–Dahab	Dense
Applicability	(2)	(2)	(2)	$\substack{m>W,\\ \deg q \le m-W}$	g fixed

$2m-1$; right-shift extraction preserves this invariant, and output padding above coefficient $m-1$ is masked explicitly. Caller-owned inputs and outputs are excluded from temporary-space counts. Section 8 separately fixes the steady-state reduction boundary, reusable-plan setup, storage, and any amortization over a stated reuse count.

3. Existing Reduction Paradigms

Serial folding repeatedly eliminates high terms through the nonleading support. It has low setup, but a tap at distance $\Delta_{\min}$ can create on the order of $m/\Delta_{\min}$ dependent folds. Fixed trinomial and pentanomial formulae, generated code, and López–Dahab reduction can be stronger under their family assumptions [10].

Dense linear maps and generic parallel division expose a different tradeoff. Polynomial division can be reduced to triangular Toeplitz or reciprocal inversion [2]. Barrett and Montgomery instead use polynomial products and modulus-dependent constants. Their machine cost therefore depends on multiplication and memory behavior as well as tapwise shift/XOR counts.

Table 1 summarizes the applicability of the direct-reduction methods used in the comparison.

The general-code winner contract compares runtime loop implementations; generated fixed-modulus code forms a separate specialization contract. Montgomery REDC likewise belongs to a representation-aware multiplication contract because it maps A to $AR^{-1} \bmod g$ instead of performing the direct reduction $A \mapsto A \bmod g$.

4. Feedback Operators

Lemma 4.1 (Low/high decomposition)**.** *For* (5) *and* (6)*,*

$$A - gY = L + V(Y) + x^m\big(H + Y + U(Y)\big). \tag{7}$$

Consequently, if $H = (I+U)Y$*, then* $A \bmod g = L + V(Y)$*.*

Proof. Using (5) and (6), and using that subtraction and addition coincide over $\mathbb{F}_2$, we obtain

$$\begin{aligned} A - gY &= L + x^m H + x^m Y + qY \\ &= L + V(Y) + x^m\big(H + Y + U(Y)\big). \end{aligned}$$

If $H = (I+U)Y$, the entire high-part factor in the last display vanishes. Hence $A-(L+V(Y)) = gY$, while $\deg(L+V(Y)) < m$. The uniqueness of the remainder on division by the monic polynomial g proves the claim. □

Lemma 4.2 (Support-nondensifying Frobenius powers)**.** *For every* $k \ge 0$*,*

$$U^{2^k} = \sum_{t \in T} N^{2^k \Delta_t}, \tag{8}$$

where shifts by at least m are zero.

Proof. The truncated shifts satisfy $N^a N^b = N^{a+b}$ and therefore commute. In the endomorphism algebra of $\mathbb{F}_2^m$, commuting elements satisfy $(X_1 + \cdots + X_s)^2 = X_1^2 + \cdots + X_s^2$. Starting from $U = \sum_{t\in T} N^{\Delta_t}$ and iterating this identity k times gives

$$U^{2^k} = \sum_{t\in T} (N^{\Delta_t})^{2^k} = \sum_{t\in T} N^{2^k \Delta_t}.$$

Finally, $N^d = 0$ for $d \geq m$. For $d < m$, the matrix of N^d occupies the dth superdiagonal. Distinct surviving shifts therefore have disjoint matrix support and cannot cancel. □

Theorem 4.3 (Frobenius-factorized feedback inverse)**.** *If $U = 0$, set $r = 0$; then $(I + U)^{-1} = I$, with the empty product understood as I. Otherwise, for*

$$r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil, \tag{9}$$

we have $U^{2^r} = 0$ and

$$(I + U)^{-1} = \prod_{k=0}^{r-1} (I + U^{2^k}). \tag{10}$$

Moreover, r is the least nonnegative integer for which $U^{2^r} = 0$.

Proof. Suppose $U \neq 0$. Then T contains a positive tap, $\Delta_{\min} < m$, and $r \geq 1$. By the definition of r, every $t \in T$ satisfies $2^r \Delta_t \geq 2^r \Delta_{\min} \geq m$. Lemma 4.2 therefore gives $U^{2^r} = 0$. On the other hand, $2^{r-1}\Delta_{\min} < m$, so the unique tap at distance $\Delta_{\min}$ contributes the nonzero shift $N^{2^{r-1}\Delta_{\min}}$ to $U^{2^{r-1}}$. Thus r is minimal.

For any $j \geq 0$, repeated use of $(I + X)^2 = I + X^2$ gives the telescoping identity

$$(I + U) \prod_{k=0}^{j-1} (I + U^{2^k}) = I + U^{2^j}.$$

Taking $j = r$ makes the right-hand side I. All factors are polynomials in U and hence commute, so the product is both a left and a right inverse of $I + U$. The case $U = 0$ is immediate. □

This is a factored truncated reciprocal. Formal-series inversion itself is classical [6]; the proposed contribution is the support-nondensifying fixed-length realization and its complete tap-geometry analysis.

5. Frobenius-Factorized Reduction

FFR applies the Frobenius factors of the truncated feedback inverse without materializing that inverse. We distinguish a *planned* realization, which retains compact shift descriptors, from an *online* realization, which derives them from the original taps on every call. Both implementation interfaces accept the complete nonleading support T as a duplicate-free list of exponents in strictly increasing order, and both constructors validate this canonical representation.

For every $k \geq 0$, define

$$T_k = \{t \in T : 2^k \Delta_t < m\}. \tag{11}$$

Then $T_k = \varnothing$ for $k \geq r$. At stage k, either realization uses the shifts $2^k\Delta_t$ for $t \in T_k$. A reusable plan stores these shifts for $0 \leq k < r$ and the taps for final low assembly. The online realization instead enumerates T_k, doubles the active distances, and constructs the same descriptors in tap order immediately before applying the stage; the descriptors are overwritten at the next stage and are not reused between reductions as a schedule.

Algorithm 1 (Frobenius-factorized reduction). Split $A = L + x^m H$ and set $Y \leftarrow H$. For $k = 0, \ldots, r-1$, read one immutable stage input Y_{old} and compute synchronously

$$Y_{\mathrm{new}} = Y_{\mathrm{old}} + \sum_{t \in T_k} N^{2^k \Delta_t} Y_{\mathrm{old}}.$$

Set $Y \leftarrow Y_{\mathrm{new}}$ and return $L + V(Y)$.

An in-place tap loop that exposes updates within a stage computes a different recurrence.

Theorem 5.1 (Correctness)**.** *For every monic $g \in \mathbb{F}_2[x]$ of degree m and every A with $\deg A < 2m$, Algorithm 1 returns $A \bmod g$.*

Proof. At stage k, every summand reads the same value Y_{old}, so the stage maps its input to

$$\left(I + \sum_{t \in T_k} N^{2^k \Delta_t}\right) Y_{\mathrm{old}} = (I + U^{2^k}) Y_{\mathrm{old}}$$

by Lemma 4.2. After all stages,

$$Y = \prod_{k=0}^{r-1} (I + U^{2^k}) H = (I + U)^{-1} H$$

by Theorem 4.3; the order of the factors is immaterial because they commute. Hence $H = (I + U)Y$, and Lemma 4.1 shows that the returned polynomial $L + V(Y)$ is $A \bmod g$. □

Example 5.2. Let

$$g = x^{16} + x^{15} + x^{11} + x^3 + 1.$$

The nonconstant taps have distances 1, 5, and 13; the constant tap has distance 16 and contributes no feedback. Thus $r = 4$, and Algorithm 1 applies the four maps

$$I + N + N^5 + N^{13}, \qquad I + N^2 + N^{10}, \qquad I + N^4, \qquad I + N^8.$$

The shifts 26, 20, and 52 have disappeared by truncation. Their mixed feedback paths have not been discarded: they arise implicitly when these four factors are composed. For the resulting state Y, the algorithm returns

$$L + Y + (x^3 Y \bmod x^{16}) + (x^{11} Y \bmod x^{16}) + (x^{15} Y \bmod x^{16}).$$

Remark 5.3 (Boundary cases). If $T = \varnothing$, then $g = x^m$, $U = V = 0$, no feedback stage is executed, and the remainder is L. If $T = \{0\}$, then $g = x^m + 1$, $U = 0$ and $V = I$, so the remainder is $L + H$. Omitting the constant tap merely removes the identity summand from V; it does not change the argument. Dense and reducible moduli satisfy the same correctness theorem, although density can make the schedule unattractive. Word misalignment affects implementation masking and storage, not the algebraic algorithm.

Appendix A gives the coefficient-algebra and odd-characteristic extensions.

6. Work–Depth–Space Geometry

We first count *scheduled coefficient contributions*: adding one shifted source coefficient to one destination coefficient costs one unit. Retaining the unshifted stage input, copying a vector, testing a public bound, and reading a schedule descriptor are not included in this measure. Let $s = |T|$, let $h_k = |T_k|$, and define

$$\ell_t = \#\{k \geq 0 : 2^k \Delta_t < m\} = \left\lceil \log_2 \frac{m}{\Delta_t} \right\rceil. \tag{12}$$

The equality includes the constant tap: if $t = 0$, then $\Delta_t = m$ and $\ell_t = 0$.

Theorem 6.1 (Geometry-sensitive scheduled work)**.** *The number of stored feedback shifts and the feedback contribution work are, respectively,*

$$S_{\mathrm{fb}} = \sum_{k=0}^{r-1} h_k = \sum_{t \in T} \ell_t, \tag{13}$$

$$W_{\mathrm{fb}} = \sum_{t \in T} \sum_{k \geq 0} [m - 2^k \Delta_t]_+, \qquad [z]_+ = \max\{z, 0\}. \tag{14}$$

The final low-part assembly has scheduled coefficient work

$$W_V = \sum_{t \in T} (m - t) = \sum_{t \in T} \Delta_t. \tag{15}$$

If $s \geq 1$, then

$$S_{\mathrm{fb}} \leq s + \log_2 \frac{m^s}{s!} \leq s \left(1 + \log_2 \frac{em}{s}\right), \tag{16}$$

$$W_{\mathrm{fb}} \leq m S_{\mathrm{fb}} = O\Big(ms \left(1 + \log \frac{m}{s}\right)\Big), \tag{17}$$

$$W_{\mathrm{fb}} + W_V = O\Big(ms \left(1 + \log \frac{m}{s}\right)\Big). \tag{18}$$

The inequality in (17) *is strict whenever $U \neq 0$.*

Proof. Tap t appears precisely for the ℓ_t integers k satisfying $2^k \Delta_t < m$, proving (13). At such an occurrence, the right shift affects exactly $m - 2^k \Delta_t$ coefficient positions. Summing these contributions proves (14). Similarly, $x^t Y \bmod x^m$ contains $m - t = \Delta_t$ scheduled source positions, which proves (15).

Order the distinct distances as $1 \leq d_1 < \cdots < d_s \leq m$. Since $d_i \geq i$ and $\lceil z \rceil \leq z + 1$,

$$S_{\mathrm{fb}} \leq \sum_{i=1}^{s} \left(1 + \log_2 \frac{m}{i}\right) = s + \log_2 \frac{m^s}{s!}.$$

The standard inequality $s! \geq (s/e)^s$ gives the second part of (16). Every one of the S_{fb} feedback shifts affects at most m positions, and affects strictly fewer than m positions when it is nonzero. This proves (17), including the non-strict boundary $W_{\mathrm{fb}} = S_{\mathrm{fb}} = 0$ for a constant-only tap set. Finally, $W_V \leq ms$, and (18) follows. □

Corollary 6.2 (Feedback depth and a gate-depth upper bound)**.** *If $U \neq 0$, Algorithm 1 has exactly*

$$r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil$$

sequential feedback stages; if $U = 0$, it has none. Thus, for fixed m and $\Delta_{\min}$, feedback depth is independent of s. In a fan-in-two XOR model with free fanout and unrestricted temporary wires, the complete reduction has the upper bound

$$D_{\oplus} \le \sum_{k=0}^{r-1} \lceil \log_2(1+h_k) \rceil + \lceil \log_2(1+s) \rceil . \tag{19}$$

Proof. The exact feedback-stage count is the minimality statement in Theorem 4.3. Within stage k, each output coefficient is the sum of its old value and at most h_k shifted values, so a balanced XOR tree has depth at most $\lceil \log_2(1+h_k) \rceil$. The stages are composed sequentially. Final assembly adds L and at most s shifted values, giving the last term in (19). □

Equation (19) is a fan-in-two Boolean-circuit upper bound. Feedback depth treats a whole synchronous vector update as one stage, whereas gate depth accounts for the XOR tree inside that stage.

6.1. **Packed-word geometry.** Let W be the machine-word width. Put $n = \lceil m/W \rceil$. The following counts describe destination-word contributions; they are not instruction counts. A feedback shift by $d < m$ has potentially nonzero coefficients in $\lceil (m-d)/W \rceil$ destination words. A low-assembly shift by t touches

$$n - \left\lfloor \frac{t}{W} \right\rfloor$$

destination words. The latter is not always $\lceil (m-t)/W \rceil$: a non-word-aligned interval may intersect an additional word.

Proposition 6.3 (Word-level loop geometry)**.** *For $U \neq 0$, set*

$$a_k = \left\lceil \frac{m - 2^k \Delta_{\min}}{W} \right\rceil \qquad (0 \le k < r).$$

The useful feedback-word support and the shifted-word contributions traversed by the scalar stage loops are, respectively,

$$\widehat{B}_{\mathrm{fb}} = \sum_{t \in T} \sum_{\substack{k \ge 0 \\ 2^k \Delta_t < m}} \left\lceil \frac{m - 2^k \Delta_t}{W} \right\rceil , \tag{20}$$

$$B_{\mathrm{fb}}^{\mathrm{loop}} = \sum_{k=0}^{r-1} \sum_{t \in T_k} \min \left\{ a_k,\, n - \left\lfloor \frac{2^k \Delta_t}{W} \right\rfloor \right\} , \tag{21}$$

$$B_V = \sum_{t \in T} \left(n - \left\lfloor \frac{t}{W} \right\rfloor \right) . \tag{22}$$

They satisfy

$$\widehat{B}_{\mathrm{fb}} \le B_{\mathrm{fb}}^{\mathrm{loop}} \le \widehat{B}_{\mathrm{fb}} + S_{\mathrm{fb}}. \tag{23}$$

Moreover,

$$\sum_{k=0}^{r-1} a_k \le B_{\mathrm{fb}}^{\mathrm{loop}}. \tag{24}$$

An in-place scalar implementation makes one n-word high-part extraction, traverses a_k destination words at feedback stage k, and makes one n-word final assembly pass, in addition to the $B_{\mathrm{fb}}^{\mathrm{loop}} + B_V$ shifted contributions.

Proof. A right shift by d occupies coefficient positions $0, \ldots, m-d-1$, which span $\lceil (m-d)/W \rceil$ words. A left shift by t occupies positions $t, \ldots, m-1$, which span the words numbered $\lfloor t/W \rfloor, \ldots, n-1$. At stage k, the smallest active shift is $2^k \Delta_{\min}$, so the union of all affected destinations spans exactly a_k words. For a particular feedback shift $d = 2^k \Delta_t$, the scalar loop also requires the source-word index $\lfloor d/W \rfloor$ to remain below the end of the state. It therefore visits

$$\min\{a_k, n - \lfloor d/W \rfloor\}$$

destinations. This is either the useful count $\lceil (m-d)/W \rceil$ or that count plus one; the latter case performs only a padding-word contribution. Summing proves (23). For each $k < r$, a tap attaining $\Delta_{\min}$ is active and contributes

$$\min\left\{a_k, n - \left\lfloor \frac{2^k \Delta_{\min}}{W} \right\rfloor\right\} = a_k.$$

Summing these contributions proves (24); therefore the complete scalar loop decomposition is $O(n + B_{\mathrm{fb}}^{\mathrm{loop}} + B_V)$. The extraction and assembly statements follow from traversing the m-bit input and output states once. □

A word-aligned contribution can be formed from one source word. A non-word-aligned contribution can require two adjacent source words, two machine shifts, and an XOR before accumulation. Consequently, Proposition 6.3 records loop geometry; the latency of aligned and cross-word contributions is ISA-specific and is determined by measurement.

Proposition 6.4 (Temporary space, online generation, and schedule setup)**.** *Excluding the caller-owned input and output, the in-place scalar reduction can be performed with one m-bit state. The implementation evaluated in this paper allocates exactly $n+1$ plan-owned scratch-array words: n state words and one zero sentinel. An explicit reusable plan stores*

$$r \text{ stage records}, \qquad S_{\mathrm{fb}} \text{ feedback-shift records}, \qquad s \text{ assembly-shift records},$$

in addition to constant-size metadata. Such a plan can be constructed in $O(s + r + S_{\mathrm{fb}})$ word-RAM operations and the same asymptotic number of records. Alternatively, the factors can be generated online with no persistent feedback or assembly schedule. This realization uses $n+1$ state words and s reusable descriptor records, hence $O(n+s)$ workspace, and performs $O(s+r+S_{\mathrm{fb}})$ additional scalar descriptor-generation operations per reduction.

Proof. For a positive right shift, output coefficient i depends only on old coefficients with indices at least i. Updating coefficients from low to high therefore never destroys a value needed by a later destination. This permits one in-place state; the extra sentinel is an implementation device for branch-free adjacent-word reads.

The record counts follow directly from (13). For the setup bound, compute every ℓ_t by repeated doubling, at total cost $O(s + S_{\mathrm{fb}})$. Because each tap is active on the stage interval $0, \ldots, \ell_t - 1$, a difference array followed by one prefix sum computes all r per-stage counts in $O(s + r)$ operations. A second pass emits the ℓ_t doubled shifts of each tap into the corresponding stage buckets in $O(S_{\mathrm{fb}})$ operations.

TABLE 2. Reduction work and dependency depth. Here $n = \lceil m/W \rceil$, $s = |T|$, and fan-in-two depth assumes free fanout. As defined in Proposition 6.3, $B_{\mathrm{fb}}^{\mathrm{loop}}$ counts shifted-word contributions traversed by the feedback loops, while B_V counts those traversed by final low-part assembly. The serial row assumes $U \neq 0$, and the López–Dahab row assumes $\deg q \leq m - W$. The displayed depths retain the natural dependency measure of each method rather than defining a common gate-depth metric.

Method	Work	Dependency depth
FFR	$C_{\mathrm{word}} = O\Big(n + B_{\mathrm{fb}}^{\mathrm{loop}} + B_V\Big)$	$D_{\mathrm{fb}} = r = \lceil \log_2(m/\Delta_{\min}) \rceil$
Serial	$C_{\mathrm{word}} = O(sn\nu_U)$	$D_{\mathrm{fb}} \leq \nu_U \leq \lceil m/\Delta_{\min} \rceil$
López–Dahab	$C_{\mathrm{word}} = \Theta(n(1+s))$	$D_{\mathrm{loop}} = n + O(1)$
BarrettGF2X	$C_{\mathrm{word}} = 2M(n) + O(n)$	$D = 2D_M(n) + O(1)$
Dense map	$C_{\mathrm{word}} = \Theta(mn) = \Theta(m^2/W)$	$D = O(\log m)$

For the online realization, keep one s-record descriptor array. Because the canonical taps are ordered, traversing them from highest to lowest emits the active feedback distances in increasing order and stops at the first inactive distance. The total number emitted is S_{fb}, with at most one termination check per stage; final assembly emits s descriptors. This gives $O(s + r + S_{\mathrm{fb}})$ scalar control work without carrying a descriptor set forward as the schedule for the next reduction. □

The setup bound describes an available two-pass schedule construction, not compilation of generated code. The benchmarked reusable-plan builder uses a simpler stage-by-stage scan, sort, and allocation strategy and is not claimed to realize this asymptotic setup bound. The experiments time that concrete builder and the schedule-free workspace separately, and measure online descriptor generation inside reduction. For comparison, serial tapwise folding needs no doubled-shift schedule but, when $U \neq 0$, can execute up to $\lceil m/\Delta_{\min} \rceil$ dependent feedback iterations; López–Dahab has $\Theta(n(1+s))$ shift/scatter work under its $m > W$ and $\deg q \leq m - W$ applicability conditions; the Barrett baseline performs two polynomial multiplications on $\Theta(n)$-word operands after reciprocal setup; and a materialized dense map stores $\Theta(mn)$ words and performs $\Theta(mn)$ row-word products. These are different cost primitives, so their constants and crossovers are determined experimentally rather than by equating the displayed source-level counts.

For a compact comparison, let $M(n)$ and $D_M(n)$ denote the work and dependency depth of an n-word polynomial multiplication. For $U \neq 0$, let $\nu_U = \min\{j \geq 1 : U^j = 0\}$; then $\nu_U \leq \lceil m/\Delta_{\min} \rceil$. The boundary $U = 0$ has no dependent feedback fold. Table 2 collects the resulting method-specific source-level costs; Section 8 supplies the corresponding measured times.

6.2. **Uniform fixed-weight supports.** We next record what the deterministic geometry implies for one explicit distribution. Fix m and $1 \leq s \leq m$, and choose T uniformly among the s-subsets of $\{0, \ldots, m-1\}$. Equivalently, the distances form

a uniform s-subset of $\{1, \dots, m\}$. This model permits a zero constant coefficient and reducible moduli; it is not a distribution on irreducible polynomials.

Theorem 6.5 (Random-support schedule geometry)**.** *Under the uniform fixed-weight model,*

$$\mathbb{E}[h_k] = \frac{s}{m}\left\lfloor \frac{m-1}{2^k} \right\rfloor < \frac{s}{2^k}, \tag{25}$$

$$\mathbb{E}[S_{\mathrm{fb}}] < 2s, \tag{26}$$

$$\mathbb{E}[r] \le \lceil \log_2 s \rceil + 2, \tag{27}$$

$$\mathbb{E}[W_{\mathrm{fb}}] < 2ms. \tag{28}$$

Moreover, $\mathbb{E}[W_V] = s(m+1)/2$*, and hence the expected total scheduled coefficient contribution work is* $O(ms)$*.*

Proof. There are exactly $\lfloor (m-1)/2^k \rfloor$ distances $d \in \{1, \dots, m\}$ with $2^k d < m$. Each distance belongs to the random support with probability s/m, proving (25). Summing over $k \ge 0$ and using $S_{\mathrm{fb}} = \sum_k h_k$ gives (26).

Since $r = \sum_{k \ge 0} \mathbf{1}_{\{h_k > 0\}}$, Markov's inequality and (25) give

$$\Pr(h_k > 0) \le \min\{1, s/2^k\}.$$

Summing the first $\lceil \log_2 s \rceil$ unit bounds and the remaining geometric tail proves (27). The deterministic inequality $W_{\mathrm{fb}} \le m S_{\mathrm{fb}}$ proves (28). Finally, every distance has inclusion probability s/m, so

$$\mathbb{E}[W_V] = \frac{s}{m} \sum_{d=1}^{m} d = \frac{s(m+1)}{2}.$$

□

Exact work depends on every tap distance beyond the summary coordinates $(h, \Delta_{\min})$. The bounds above therefore describe families and schedules, while method selection at fixed $(m, h, \Delta_{\min})$ can still vary with the complete tap set.

7. Prior Art and Novelty Boundary

Reciprocal and triangular Toeplitz inversion for polynomial division are classical [2], as are formal-power-series inversion [6]. Parallel triangular Toeplitz and recurrence solvers already use doubling, bandwidth, or prefix structure [3, 13, 9, 7]. These works establish the classical status of finite geometric inversion and logarithmic-depth recurrence solving. Our question is whether a support-nondensifying fixed-state realization retains useful support geometry without dense reciprocals, polynomial multiplication, or expanded state.

The direct predecessor of the present method is our Suwako algorithm for trinomials $x^m + x^t + 1$ [1, Lemma 2, Eq. (5)]. It already replaces the $\Theta(m/(m-t))$ serial feedback chain by logarithmically many doubling steps without modulus-specific precomputation. The present paper does not claim that trinomial mechanism anew: it derives the multi-tap operator equation, shows that every Frobenius factor contains only the surviving doubled images of the original taps, and develops exact geometry and packed implementations for arbitrary monic binary moduli.

The closest external recurrence-level antecedent is the term-preserving look-ahead transformation for LFSRs: in characteristic two, replacing a generator by

its square doubles the feedback distances without increasing its term count [11]. Parallel LFSR and CRC architectures also construct equivalent state-space updates for arbitrary generator polynomials [4]. These provide the nearest recurrence-level antecedents for term-preserving doubling and parallel feedback. They transform or parallelize a homogeneous recurrence, whereas Algorithm 1 applies successive factors of an implicit inverse to a forced length-m triangular system without enlarging the state.

Sparse binary-field reduction includes word- and bit-parallel polynomial-basis methods [16] and strong specialized formulae [10]. Niehues, Custodio, and Panario give a uniform top-down reducer for arbitrary low-weight moduli and explicitly observe that tap placement can induce a long critical path [14]. Finite-field multiplier architectures additionally use multi-degree reduction, subexpression sharing, and balanced XOR trees [12]. Our measured comparison accordingly keeps serial folding and applicable López–Dahab reduction as direct baselines.

Barrett and Montgomery provide multiplication-based alternatives, including restricted precomputation-free families [8]. FFR differs in supporting arbitrary monic binary moduli without a materialized reciprocal or dense matrix: its planned realization has a separately measured compact schedule, while its online realization retains only reusable state and descriptor scratch and generates shifts inside each reduction.

Repeated Frobenius powering in factorization gives the closest application context. Von zur Gathen and Gerhard combine theory, implementation, storage, and complete factorization over $\mathbb{F}_2$ [5]; irreducible polynomial construction gives another context [15].

Relative to these antecedents, FFR combines arbitrary monic-modulus correctness, support-nondensifying Frobenius factors, logarithmic feedback depth, execution without a materialized dense reciprocal or matrix, and exact geometry from all tap distances.

8. Implementations and Evaluation

8.1. **Implementations and validation.** The native interface exposes planned and online FFR, serial top-down folding, and gf2x-backed Barrett reduction through the same operation

$$A \longmapsto A \bmod g, \qquad \deg A < 2m.$$

Both FFR realizations use the same scalar feedback-stage and final-assembly kernels; only the lifetime of their shift descriptors differs. FFR and serial folding use the same packed-word shift/XOR primitive. The frozen phase diagram retains `GS` (Generalized Suwako) as the historical internal identifier for planned FFR; the paper-facing names are FFR and `OnlineFFR`. We also implemented the ordinary-loop López–Dahab reducer, using it only when $\Delta_{\min} \geq W$, and a packed dense linear map for diagnostic screening. An independent long-division implementation serves exclusively as the correctness oracle. Montgomery REDC is assigned to the separate representation-aware multiplication contract described in Section 3.

Before timing, the binary implementations are compared with independent long division at word boundaries and on deterministic random instances. Separate falsification suites test the algebraic identities over $\mathbb{F}_2$, $\mathbb{F}_4$, and $\mathbb{F}_2[\varepsilon]/(\varepsilon^2)$, together with the odd-characteristic sign convention over $\mathbb{F}_3$.

Appendix B records the detailed validation and reproduction entry points.

8.2. **Experimental contract.** We measured portable scalar C on one recorded x86-64 host, pinned to logical CPU 0 and compiled with GCC 16.1.1 using `-O3 -std=c11 -Wall -Wextra`; Barrett linked `/usr/lib/libgf2x.so.3.0.0`. Every row records the governor, energy-performance preference, and turbo state, and conclusions are scoped to this platform.

Each support uses eight materialized inputs $A = L + x^m H$ with independent uniform m-bit halves. After one warm-up, 31 or 127 complete trials use twelve batch repetitions and cyclic method order. Only `reduce_into` is timed; setup, input generation, allocation, checking, and output handling are excluded, and no observation is removed.

The controlled corpus contains 1,096 constant-free supports at

$$m \in \{128, 512, 2048, 8192, 32768, 131072\}.$$

It varies h and $\Delta_{\min}$ independently by fixing the nearest tap at $m - \Delta_{\min}$ and spreading the others deterministically. Every point compares FFR with Barrett; 392 initial or screening points also include Serial, and López–Dahab is present exactly at measured points with $\Delta_{\min} \geq 64$. Winners range only over methods measured at that point.

Each method is summarized by its trial median and a deterministic 10,000-resample bootstrap 95% interval. A unique winner requires relative median-interval half-width at most 1%, a margin of at least 1% over every competitor, and paired ratio intervals strictly below one; failures remain timing instability, operational ties, or statistical ties.

8.3. **Measured winner regions.** Figure 1 gives the complete measured phase diagram. Its axes are the two principal geometric parameters rather than ordinal grid indices. Every block is one observed support; blank regions were not measured and are not interpolated. To expose the predicted regime decomposition, we overlay a fitted cost model. At each fixed m, a deterministic alternating split of the sorted grid fits affine times for FFR from its portable scalar block-work count and for López–Dahab from its shift/scatter count; Barrett is represented by its fixed-m calibration median. More precisely, the predictors and fitted times are

$$\begin{aligned} C_{\mathrm{FFR}} &= \widehat{B}_{\mathrm{fb}} + B_V, & \widehat{\tau}_{\mathrm{FFR}} &= \alpha_{\mathrm{FFR},m} + \beta_{\mathrm{FFR},m} C_{\mathrm{FFR}}, \\ C_{\mathrm{LD}} &= n(h-1) = ns, & \widehat{\tau}_{\mathrm{LD}} &= \alpha_{\mathrm{LD},m} + \beta_{\mathrm{LD},m} C_{\mathrm{LD}}, \\ & & \widehat{\tau}_{\mathrm{B}} &= b_m, \end{aligned} \tag{29}$$

where each α and β is fitted on the calibration half and b_m is the Barrett calibration median. Here C_{FFR} is a useful-block-work predictor, not the exact executed loop count $B_{\mathrm{fb}}^{\mathrm{loop}} + B_V$; their difference is the padding contribution bounded in Proposition 6.3. If $\mathcal{A}(p)$ is the set of methods available at grid point $p = (h, \Delta_{\min})$, its predicted region for method j is defined by

$$\mathcal{R}_j = \{p : \widehat{\tau}_j(p) \leq \widehat{\tau}_i(p) \text{ for every } i \in \mathcal{A}(p)\}. \tag{30}$$

The minimum predicted time labels each measured coordinate, and a line is drawn only where adjacent coordinates belong to different predicted regions. Thus the blocks remain measurements whereas the lines are discrete, descriptive approximations to the pairwise interfaces induced by (30).

Of the 1,096 cells, 983 have a unique winner: FFR wins 321, Barrett wins 305, and López–Dahab wins 357. The remaining 113 cells comprise 103 timing-unstable

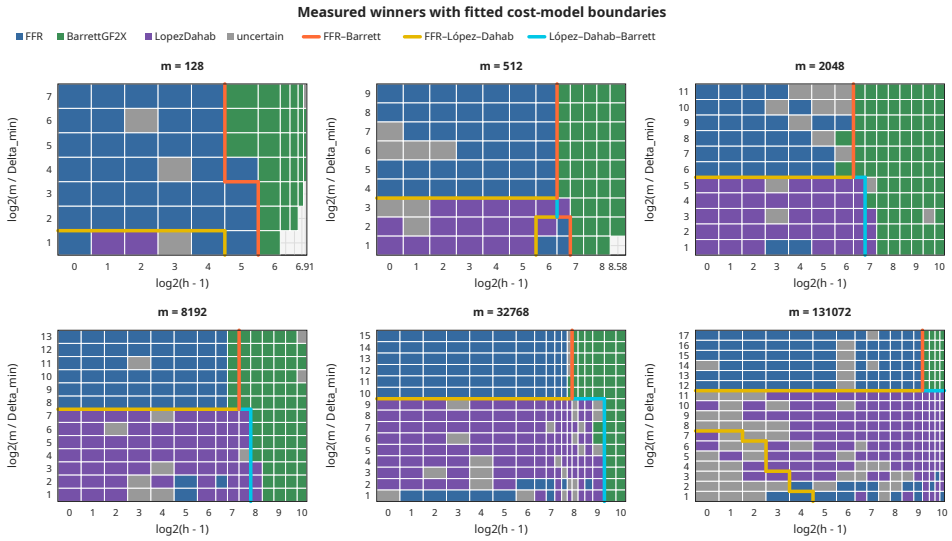

FIGURE 1. Measured winner regions for six fixed degrees. The horizontal axis is $\log_2(h-1)$ and the vertical axis is $\log_2(m/\Delta_{\min})$. Colors identify the unique winner among the methods available at that point; gray cells are uncertain. FFR and Barrett are present everywhere, whereas López–Dahab is admitted only at its tested $\Delta_{\min} \geq 64$ boundary. Thick lines mark pair-specific interfaces predicted by the fixed-m model (29), separating regions defined by the inequalities (30). The lines connect only adjacent measured coordinates and leave the measured classifications unchanged.

cells and 10 operational ties; no cell is classified as a statistical tie. The largest panel, $m = 131072$, deliberately retains only 31 trials and consequently has 52 unstable cells out of 289; these cells remain unassigned in the phase diagram.

The diagram separates three regimes. FFR occupies the sparse, difficult-feedback region. As weight increases, multiplication-based Barrett eventually wins even when $\Delta_{\min}$ is small. When $\Delta_{\min} \geq 64$, López–Dahab increasingly dominates the friendly-feedback region as m grows. Across rows with $\Delta_{\min} < 64$, the observed persistent Barrett onset is $h = 33$–65, 97, 65–97, 129, 225–257, and 641 for the six successive degrees. In the López–Dahab-applicable rows the corresponding onsets are 65, 129, 129–193, 257–385, 513–769, and beyond the largest sampled weight 1025. These are the sampled crossover locations on the recorded platform.

Table 3 reports representative reduction times and the persistent Barrett crossovers on the $\Delta_{\min} = 1$ slice.

These medians use `portable-scalar-c:v1`, `reduction-steady-state:v1`, and the `gf2x:v1` multiplication backend on the recorded primary machine.

8.4. **Does the work model predict FFR time?** Figure 2 compares the formal scheduled coefficient work W_{fb} with measured FFR time. Within each fixed degree, the Spearman correlation ranges from 0.9868 to 0.9934. Thus the complete tap geometry captured by W_{fb} orders these controlled scalar runtimes unusually

TABLE 3. Portable-C reduction summary on the controlled constant-free $\Delta_{\min} = 1$ slice. FFR time and the first ratio use $h = 9$. The crossover is the first stable Barrett winner after which every later stable sampled weight is also Barrett; uncertain cells are omitted from this persistence test.

m	FFR (ns)	Barrett/FFR	crossover h	ratio at crossover
128	64.1	2.164	33	0.881
512	272.9	3.807	97	0.718
2048	1079.2	4.247	97	0.664
8192	4390.2	8.654	129	0.819
32768	17008.4	16.768	257	0.881
131072	68167.1	37.712	641	0.886

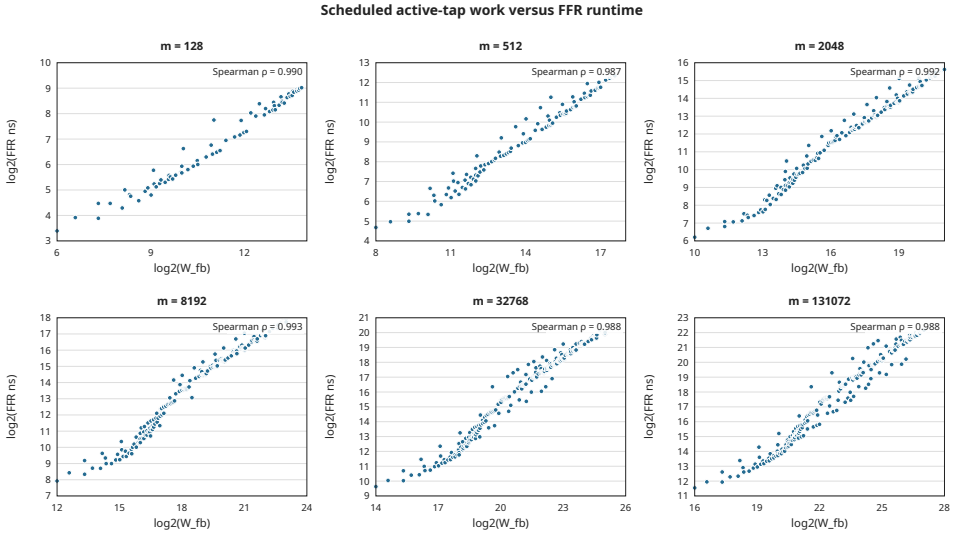

FIGURE 2. Scheduled active-tap work versus median FFR reduction time for all 1,096 controlled supports, shown separately at fixed m. Both axes use base-two logarithms. The displayed ρ is the within-panel Spearman correlation.

well. The residual scatter records effects beyond the source-work model, while the correlation summarizes runtime ordering under that model.

8.5. **Setup and storage.** Table 4 separates measured setup time from requested plan-owned storage for the three principal implementations.

Setup and requested plan-owned storage follow separately frozen contracts. The storage counts exclude allocator overhead, shared modulus storage, benchmark buffers, and transient library workspace. The setup measurements show a tradeoff

TABLE 4. Median setup time and requested plan-owned bytes at $(h, \Delta_{\min}) = (9, 1)$. Each entry is "nanoseconds / bytes."

m	trials	FFR	Serial	Barrett
128	127	596.5 / 744	119.5 / 416	782.0 / 184
512	127	719.5 / 888	123.5 / 512	4770.5 / 376
2048	127	887.5 / 1176	137.0 / 896	50799.0 / 1144
8192	127	1152.0 / 2040	149.0 / 2432	558369.5 / 4216
32768	127	1312.0 / 5208	235.0 / 8576	7760258.5 / 16504
131072	31	2832.5 / 17592	856.5 / 33152	119030566.0 / 65656

hidden by steady-state timings: Barrett's reciprocal construction becomes expensive at large m, while FFR retains a modest schedule-construction cost. For an explicitly chosen reuse count K, the artifact separately derives

$$T_{\text{total}}(K) = T_{\text{setup}} + KT_{\text{reduce}};$$

amortized values are reported only for explicitly stated reuse counts K.

To isolate whether the FFR schedule itself is necessary, we additionally ran 45 representative constant-free supports at $m \in \{128, 2048, 32768, 131072\}$, with $\Delta_{\min} \in \{1, 64, m/4\}$ and weights up to 513 where feasible. Planned and online FFR received the same eight inputs, used the same scalar kernels, and were cyclically ordered over 31 retained trials; the online path regenerated all descriptors inside the timed reduction. The median $T_{\text{online}}/T_{\text{planned}}$ over each degree slice was 1.676, 1.086, 1.006, and 1.002, respectively. Thus descriptor generation is material at small degree but is within about one percent of planned steady-state time in the two largest slices; five individual paired intervals contain one. If setup and one reduction are combined as the declared derived quantity $T_{\text{setup}} + T_{\text{reduce}}$, online FFR is lower at all 45 points and has overall median ratio 0.915. This $K = 1$ value is derived from the separately measured components. On this portable-C platform, online descriptor generation therefore approaches planned FFR performance at large degrees, while online setup still includes workspace allocation.

8.6. **End-to-end irreducibility testing.** We finally test whether the reduction result survives inside one complete computational-algebra workload. For a power-of-two degree m, Rabin's criterion computes the chain of m modular squares beginning at x, checks

$$\gcd\bigl(x^{2^{m/2}} - x, g\bigr) = 1, \qquad x^{2^m} = x \pmod{g}.$$

We deterministically selected one Sage-certified irreducible modulus at each $m \in \{128, 512, 2048, 8192\}$, always with $(h, \Delta_{\min}) = (9, 1)$. Each matched path uses the same scalar bit-dilation squarer and NTL GCD code and changes only the reducer. Its primary interval includes reducer setup, all modular squares, the GCD, and the final equality test. NTL `IterIrredTest` is timed separately as an optimized complete-library baseline.

Figure 3 shows a stable end-to-end effect. Across increasing m, NTL takes 1.35, 2.31, 3.64, and 8.04 times the FFR time; the matched Barrett path takes 1.60, 3.76, 3.27, and 6.81 times the FFR time. All paired ratio intervals lie strictly above one.

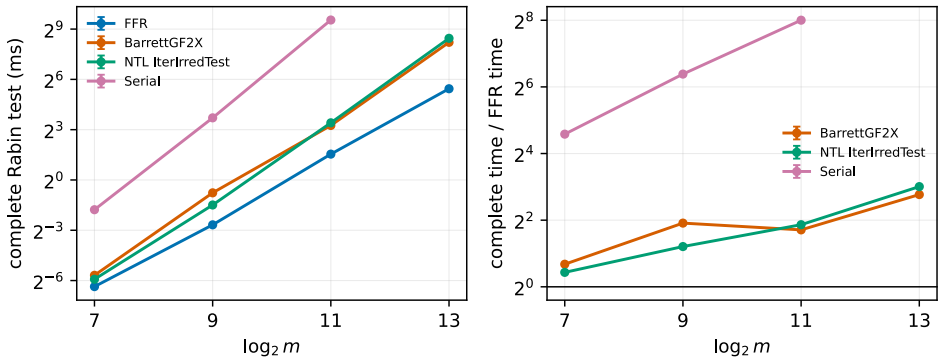

FIGURE 3. Complete Rabin irreducibility testing on four deterministically selected, Sage-certified sparse moduli with $(h, \Delta_{\min}) = (9, 1)$. The left panel shows median end-to-end time with bootstrap 95% intervals; the right panel normalizes each complete implementation by FFR. Every point contains 31 trials. Serial has no $m = 8192$ point because a retained smoke trial took about 47.5 seconds.

The modular-square chain accounts for 85.5% of FFR time at $m = 128$ and 98.9% at $m = 8192$, so the measured difference remains central to the complete test rather than being hidden by setup or the terminal GCD.

9. CONCLUSION

We recast reduction by an arbitrary monic binary polynomial as inversion of a nilpotent feedback operator. Characteristic-two Frobenius powers yield exact feedback depth $\lceil \log_2(m/\Delta_{\min}) \rceil$ and work governed by every tap distance without materializing a reciprocal or dense reduction matrix. The doubled shifts may be retained in a reusable plan or generated online with $O(n + s)$ workspace and no persistent modulus-specific schedule. The analysis separates coefficient work, packed-word loop geometry, descriptor-generation work, setup, and temporary space.

The portable scalar experiments show that this tradeoff has nontrivial algorithm-selection regions. FFR wins for sparse moduli with difficult feedback, gf2x-backed Barrett takes over as weight grows, and ordinary-loop López–Dahab is often strongest when its feedback-distance condition applies. The formal work measure is strongly monotone with FFR runtime on the controlled corpus, but the measured boundaries retain unstable cells and are scoped to the recorded platform. Representative slices further show that online descriptor generation is visible at small degree but approaches planned FFR steady-state time at the largest measured degrees. On the four certified high-tap moduli, the reduction advantage survives a complete Rabin irreducibility test and widens relative to NTL over the measured degree range, providing a concrete end-to-end computational-algebra result.

Software and Data Availability

The implementation, deterministic manifests, raw benchmark data, and reproduction scripts underlying the reported results are archived in the `moc-artifact-v3` GitHub release.

Use of Artificial Intelligence

OpenAI Codex (GPT-5.6 Sol) was used during the research and manuscript-preparation workflow to assist with literature-search planning and synthesis, mathematical exploration and statement auditing, software and test development, benchmark-analysis and plotting code, and manuscript drafting and editing. The authors reviewed the resulting sources, arguments, code, and reported data and assume full responsibility for the manuscript.

Appendix A. Coefficient-Algebra Extension

Let R be a commutative characteristic-two algebra and let

$$g = x^m + \sum_{t=0}^{m-1} a_t x^t \in R[x]$$

be monic. Define N as before on R^m, and put

$$U = \sum_{t=0}^{m-1} a_t N^{m-t}, \qquad V(Y) = \sum_{t=0}^{m-1} a_t (x^t Y \bmod x^m).$$

Theorem A.1 (Coefficient-algebra factorization)**.** *For every $k \geq 0$,*

$$U^{2^k} = \sum_{t=0}^{m-1} a_t^{2^k} N^{2^k(m-t)}. \tag{31}$$

If $U^{2^r} = 0$, then

$$(I+U)^{-1} = \prod_{k=0}^{r-1} (I + U^{2^k}).$$

Consequently, applying these factors to H and returning $L+V(Y)$ computes A mod g for every $A = L + x^m H$ of degree below $2m$.

Proof. The summands $a_t N^{m-t}$ commute. Frobenius is therefore additive on their sum, and iterating it gives (31). The telescoping identity

$$(I+U)\prod_{k=0}^{r-1}(I+U^{2^k}) = I + U^{2^r} = I$$

proves the inverse formula. Finally,

$$A - gY = L + V(Y) + x^m\big(H + Y + U(Y)\big)$$

in characteristic two, so $H = (I+U)Y$ leaves the unique degree-below-m remainder $L + V(Y)$. □

This extension preserves the formal stage schedule; its actual nonzero support may shrink. In a nonreduced algebra a coefficient may satisfy $a_t \neq 0$ and $a_t^{2^k} = 0$; for example, $\varepsilon^2 = 0$ in $\mathbb{F}_2[\varepsilon]/(\varepsilon^2)$. Thus the exact binary expression W_{fb} is scheduled work in this setting, while an implementation that removes zero coefficients may do less arithmetic. In odd prime characteristic p, the corresponding radix-p factorization uses $\sum_{j=0}^{p-1}(-U^{p^k})^j$, and the final remainder is $L - V(Y)$ rather than the characteristic-two expression $L + V(Y)$.

Appendix B. Validation and Additional Results

The deterministic algebraic suites use independent polynomial long division or direct finite geometric series as their oracles. Over $\mathbb{F}_2$, 20,000 random cases with $m \leq 128$ and all 299,592 monic-modulus/input pairs for $m \leq 6$ pass. Over $\mathbb{F}_2[\varepsilon]/(\varepsilon^2)$, 20,000 random cases and all 266,304 cases for $m \leq 3$ pass; 13,573 random instances exhibit a nonzero coefficient killed by a Frobenius power, motivating the support qualification above. A further 20,000 random $\mathbb{F}_4$ cases and 10,000 random $\mathbb{F}_3$ cases agree with independent inverse and reduction paths. Exhaustive enumeration of all 262,125 nonempty constant-free supports for $2 \leq m \leq 18$ finds no counterexample to the scheduled-work formula or its coarse bound.

The repository entry point `make check` runs these suites together with the native differential, boundary, masking, dense, and López–Dahab checks; `make check-sanitize` separately runs sanitizer builds of the native reducers and generated-reducer checks. The exact artifact entry points are

```
make artifact-cost-model; make artifact-paper; make artifact-rabin;
make artifact-microbenchmark ARTIFACT_CPU=0; make artifact-online.
```

The 67,576-row primary dataset and its 26 derived outputs are hash-locked. From a detached clean checkout at commit `8b21c90`, all 26 outputs, the 465-row Rabin artifact, and the 1,395-row, 45-support online artifact reproduced with exact hash agreement while the full `make check` and `make check-sanitize` suites passed. Every retained primary-dataset and Rabin row records its seed, taps, command, compiler and gf2x paths, binary digest, affinity, and frequency policy. The online artifact records its applicable execution metadata, including command, compiler flags, binary digest, affinity, and frequency policy; its multiplication backend is not applicable. This version contains no secondary-platform result.

References

[1] J. Zhou, J. Wang, H. Ren, S. Gao, and X. Lan, *Suwako: A logarithmic-depth modular reduction for arbitrary trinomials over $\mathbb{F}_{2^m}$ without pre-computation*, Cryptology ePrint Archive, Paper 2025/2303, 2025, https://eprint.iacr.org/2025/2303.

[2] D. Bini and V. Pan, *Fast parallel polynomial division via reduction to triangular Toeplitz matrix inversion and to polynomial inversion modulo a power*, Inform. Process. Lett. **21** (1985), no. 2, 79–81, https://doi.org/10.1016/0020-0190(85)90037-7.

[3] D. Bini, *Parallel solution of certain Toeplitz linear systems*, SIAM J. Comput. **13** (1984), no. 2, 268–276, https://doi.org/10.1137/0213019.

[4] M. Ayinala and K. K. Parhi, *High-speed parallel architectures for linear feedback shift registers*, IEEE Trans. Signal Process. **59** (2011), no. 9, 4459–4469, https://doi.org/10.1109/TSP.2011.2159495.

[5] J. von zur Gathen and J. Gerhard, *Polynomial factorization over $\mathbb{F}_2$*, Math. Comp. **71** (2002), no. 240, 1677–1698, https://doi.org/10.1090/S0025-5718-02-01421-7.

[6] K. S. K. Kalorkoti, *Inverting polynomials and formal power series*, SIAM J. Comput. **22** (1993), no. 3, 552–559, https://doi.org/10.1137/0222037.

[7] C.-W. Ho and R. C. T. Lee, *A parallel algorithm for solving sparse triangular systems*, IEEE Trans. Comput. **39** (1990), no. 6, 848–852, https://doi.org/10.1109/12.53610.

[8] M. Knezevic, K. Sakiyama, J. Fan, and I. Verbauwhede, *Modular reduction in* GF(2^n) *without pre-computational phase*, WAIFI 2008, LNCS 5130, Springer, 2008, pp. 77–87, https://doi.org/10.1007/978-3-540-69499-1_7.

[9] P. M. Kogge and H. S. Stone, *A parallel algorithm for the efficient solution of a general class of recurrence equations*, IEEE Trans. Comput. **C-22** (1973), no. 8, 786–793, https://doi.org/10.1109/TC.1973.5009159.

[10] J. López and R. Dahab, *High-speed software multiplication in* $\mathbb{F}_{2^m}$, Progress in Cryptology—INDOCRYPT 2000, LNCS 1977, Springer, 2000, pp. 203–212, https://doi.org/10.1007/3-540-44495-5_18.

[11] J.-C. Lin, S.-J. Chen, and Y. H. Hu, *Cycle-efficient LFSR implementation on word-based microarchitecture*, IEEE Trans. Comput. **62** (2013), no. 4, 832–836, https://doi.org/10.1109/TC.2012.14.

[12] P. K. Meher, *Systolic and non-systolic scalable modular designs of finite field multipliers for Reed–Solomon codec*, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. **17** (2009), no. 6, 747–757, https://doi.org/10.1109/TVLSI.2008.2006080.

[13] M. Morf, *Doubling algorithms for Toeplitz and related equations*, Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing, 1980, pp. 954–959, https://doi.org/10.1109/ICASSP.1980.1171074.

[14] L. Boppre Niehues, R. Custodio, and D. Panario, *Fast modular reduction and squaring in* GF(2^m), Inform. Process. Lett. **132** (2018), 33–38, https://doi.org/10.1016/j.ipl.2017.12.002.

[15] V. Shoup, *New algorithms for finding irreducible polynomials over finite fields*, Math. Comp. **54** (1990), no. 189, 435–447, https://doi.org/10.1090/S0025-5718-1990-0993933-0.

[16] H. Wu, *Low complexity bit-parallel finite field arithmetic using polynomial basis*, Cryptographic Hardware and Embedded Systems—CHES 1999, LNCS 1717, Springer, 1999, pp. 280–291, https://doi.org/10.1007/3-540-48059-5_24.

Sichuan University
Email address: helloworld@junyu33.me

Tsinghua University
Email address: kaiyizhang@tsinghua.edu.cn